Although `if` statements are fairly common in Ruby applications, newcomers to the Ruby language are often surprised to find that looping constructs are rarely used. *Blocks* and *iterators* often take their place.

Blocks and Iterators

Code blocks are chunks of code between braces or between `do`…`end`. A common convention is that people use braces for single-line blocks and `do`/`end` for multiline blocks:

```
{ puts "Hello" }       # this is a block

do                     ###
  club.enroll(person)    # and so is this
  person.socialize       #
end                    ###
```

To pass a block to a method, place the block after the parameters (if any) to the method. In other words, put the start of the block at the end of the source line containing the method call. For example, in the following code, the block containing `puts "Hi"` is associated with the call to the `greet` method:

```
greet  { puts "Hi" }
```

If a method call has parameters, they appear before the block:

```
verbose_greet("Dave", "loyal customer")  { puts "Hi" }
```

A method can invoke an associated block one or more times by using the Ruby `yield` statement. You can think of `yield` as being something like a method call that calls out to the block associated with the method containing the `yield`. You can pass values to the block by giving parameters to `yield`. Within the block, you list the names of the arguments to receive these parameters between vertical bars (`|`).

Code blocks appear throughout Ruby applications. Often they're used in conjunction with iterators—methods that return successive elements from some kind of collection, such as an array:

```
animals = %w( ant bee cat dog elk )    # create an array
```